Oliwier Stręk

Zadanie numeryczne NUM4

Omówienie zagadnienia

Zadanie polegało na rozwiązaniu równania $Ay = b$ dla macierzy

$$A = \begin{pmatrix} 12 & 8 & 1 & 1 & \dots & 1 & 1 & 1 & 1 \\ 1 & 12 & 8 & 1 & \dots & 1 & 1 & 1 & 1 \\ 1 & 1 & 12 & 8 & \dots & 1 & 1 & 1 & 1 \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ 1 & 1 & 1 & 1 & \dots & 1 & 12 & 8 & 1 \\ 1 & 1 & 1 & 1 & \dots & 1 & 1 & 12 & 8 \\ 1 & 1 & 1 & 1 & \dots & 1 & 1 & 1 & 12 \end{pmatrix}$$

oraz wektora

$$b \equiv (5, \quad \dots, \quad 5)^T$$

Wymiar macierzy został ustalony na $N = 80$.

Algorytm zaimplementowałem samodzielnie, używając wzoru Shermana-Morrisona. Dodatkowo macierz A przechowywałem w mniejszej macierzy w rozmiarze $2 \times N$.

Po dokonaniu całości obliczeń wynik sprawdzam przy użyciu biblioteki Eigen.

Wynik dla $N = 80$

Po uruchomieniu programu dla parametru $N = 80$ – wykorzystując moją implementację – otrzymałem następujący wektor y, który jest rozwiązaniem równania $Ay = b$.

$y = (0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508188, 0.0508187,$
$0.0508188, 0.0508187, 0.0508188, 0.0508187, 0.0508188, 0.0508186, 0.050819, 0.0508183,$
$0.0508194, 0.0508178, 0.0508203, 0.0508163, 0.0508226, 0.0508127, 0.0508282, 0.0508039,$
$0.0508421, 0.050782, 0.0508765, 0.050728, 0.0509614, 0.0505946, 0.0511709, 0.0502653,$
$0.0516885, 0.0494521, 0.0529664, 0.0474439, 0.0561221, 0.0424849, 0.0639148, 0.0302393,$
$0.0831579)^T$

Dla możliwości porównania wyników podaję wektor y – obliczony dla tych samych danych – za pomocą narzędzi, które daje biblioteka Eigen.

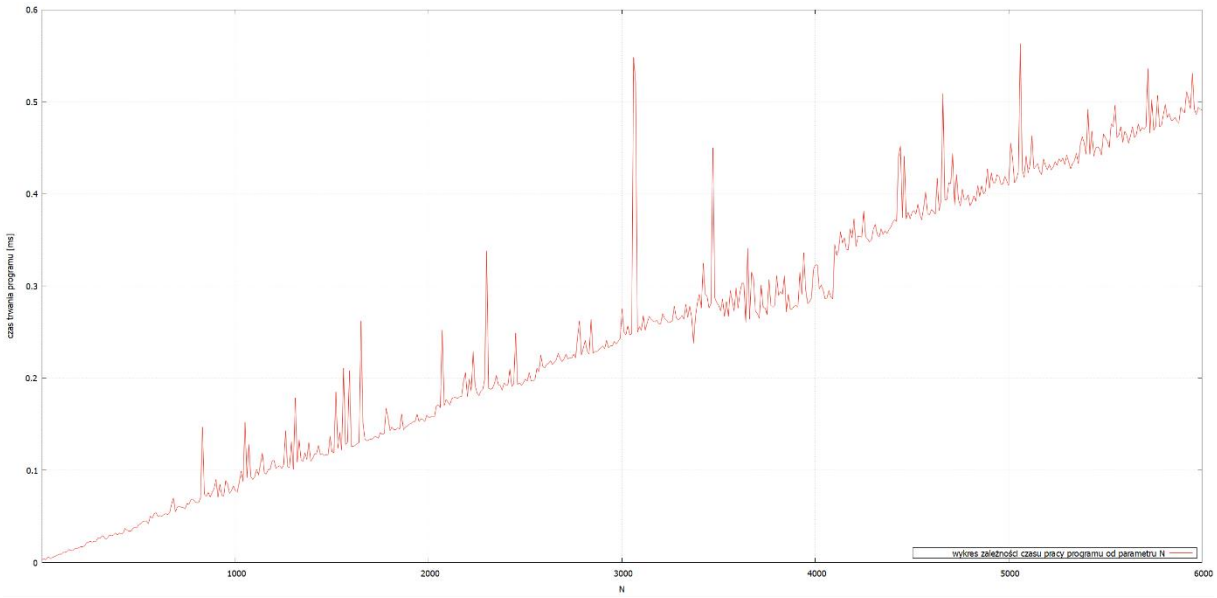
$y = (0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187,$
$0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508187, 0.0508188, 0.0508187,$
$0.0508188, 0.0508187, 0.0508188, 0.0508187, 0.0508188, 0.0508186, 0.050819, 0.0508183,$
$0.0508194, 0.0508178, 0.0508203, 0.0508163, 0.0508226, 0.0508127, 0.0508282, 0.0508039,$
$0.0508421, \ 0.050782, 0.0508765, 0.050728, 0.0509614, 0.0505946, 0.0511709, 0.0502653,$
$0.0516885, 0.0494521, 0.0529664, 0.0474439, 0.0561221, 0.0424849, 0.0639148, 0.0302393,$
$0.0831579)^T$

Wnioski

Moja implementacja i zastosowanie wzoru Shermana-Morrisona zapewnia identyczne wyniki obliczeń jak biblioteka Eigen, stwierdzam więc, że zaimplementowałem algorytm dość dobrze.

Wnioski dla N jako zmiennej

W moim programie jako maksymalny rozmiar macierzy przyjąłem $N_{max} = 6001$, w pętli *for* iterowałem co wartość 10. Podczas każdej iteracji mierzyłem czas wykonywania się obliczeń dla danej macierzy (czas na inicjalizację zmiennych oraz ustawienie odpowiednich wartości został pominięty). Następnie dane wyeksportowałem do pliku *.dat* i za pomocą programu *gnuplot* narysowałem poniższy wykres.



Na wykresie widać jak zmienia się czas trwania obliczeń maszyny cyfrowej wraz ze wzrostem parametru N.

Wnioski

Zależność czasu wykonywania się algorytmu od parametru N jest liniowa.

„Szumy”, które pojawiają się na wykresie mogą wynikać z błędu zaokrągleń i ograniczonej dokładności zmiennych typu *double*.

Podsumowanie

Przy odpowiednio dobranym sposobie na poradzenie sobie z dość specyficzną macierzą, jesteśmy w stanie sprowadzić dość kosztowny problem algorytmiczny do złożoności $O(N)$ – zależności liniowej.

Wnioski

Efektywne dobranie sposobu na rozwiązanie problemu na maszynie cyfrowej jest niezwykle ważne – pozwala ono na dużą optymalizację złożoności problemu.